



UNIVERSITÀ DEGLI STUDI
DI NAPOLI FEDERICO II

UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II

CORSO DI LAUREA IN INGEGNERIA INFORMATICA

Progetto di Network and Cloud Infrastructures

Studente:

ERNESTO CIFUNI

Matr. DE9000086

Anno Accademico 2025/2026

Indice

1	Installazione	2
2	Creazione Topologia	2
3	Topology Slicing	2
3.1	Definizione della tabella statica delle regole	2
3.2	Helper Function	3
3.3	Modifica del meccanismo di MAC learning	3
3.4	Testing	4
3.4.1	Comando Pingall	4
3.4.2	Percorso dei pacchetti nelle slice corrette	4
4	Service Slicing	4
4.1	Strutture dato per gli switch	5
4.2	Funzion Helper	5
4.3	Logica di Slicing	6
4.4	Testing	8
4.4.1	Connettività di base	8
4.4.2	Instradamento traffico video	8

1 Installazione

Come riportato nel materiale, sono stati scaricati i seguenti pacchetti per il corretto setup dell'ambiente di emulazione:

```
sudo apt install git
sudo apt install python3-pip
sudo pip3 install pandas
pip install ryu
sudo apt install d-itg
sudo apt install nload
```

```
sudo apt-get install mininet
```

2 Creazione Topologia

La topologia della rete scelta è illustrata qui di seguito:

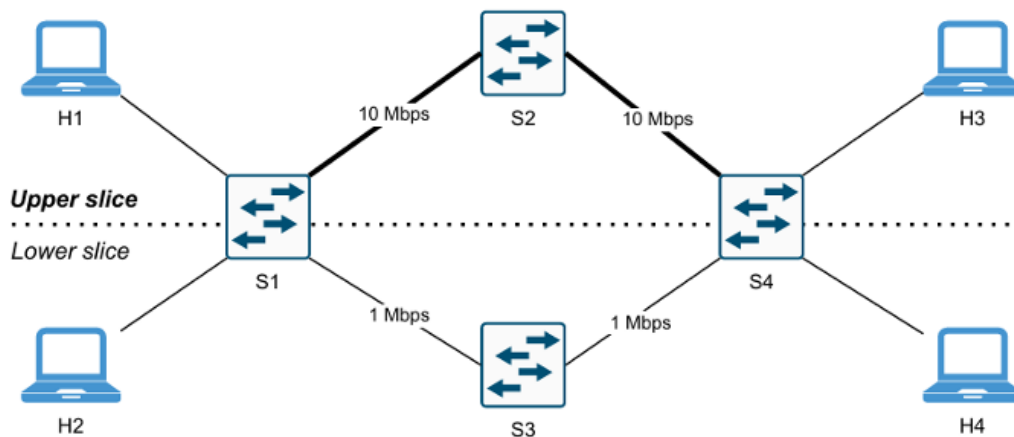


Figura 1: Topologia di rete

L'esecuzione del comando *pingall* mostra come, senza implementare tecniche di slicing, si verifichi una tempesta di broadcast.

3 Topology Slicing

Le modifiche introdotte al file originale (contenente l'implementazione del `simple_switch_13` di Ryu) si articolano in 3 punti principali:

3.1 Definizione della tabella statica delle regole

Ho definito un dizionario python che mappasse ogni switch (identificato da un datapath id) alle proprie regole di instradamento, definite in modo statico.

```
1 SLICE_TABLE = {
2     1: {1: 3, 3: 1, 2: 4, 4: 2},
```

```

3         2: {1: 2, 2: 1},
4         3: {1: 2, 2: 1},
5         4: {1: 3, 3: 1, 2: 4, 4: 2},
6     }

```

Le associazioni del dizionario seguono la logica: dpid: in_port: out_port

3.2 Helper Function

Ho poi definito una funzione helper per ritornare la porta di uscita imposta dalla slice, oppure None se lo switch/porta non e' previsto nella topologia:

```

1 def _get_slice_out_port(self, dpid, in_port):
2     return self.SLICE_TABLE.get(dpid, {}).get(in_port)

```

3.3 Modifica del meccanismo di MAC learning

Dato lo slicing statico che si vuole imporre alla topologia, ho modificato il metodo `_packet_in_handler` (invocato all'atto della ricezione di un pacchetto per cui non si conosce già la regola di bridging), rimosso la logica che prevedeva di ricorrere al flooding¹ e sostituito il tutto con una logica che deriva la porta di uscita invocando la funzione helper menzionata precedentemente. Di seguito parte della logica aggiunta:

```

1         out_port = self._get_slice_out_port(dpid, in_port)
2
3         if out_port is None:
4             self.logger.warning(
5                 "Nessuna regola di slicing per dpid=%s
6                 in_port=%s: scarto il pacchetto",
7                 dpid, in_port)
8             return
9
10        actions = [parser.OFPActionOutput(out_port)]
11        #Memorizzazione regola
12        match = parser.OFPMatch(in_port=in_port, eth_src=src,
13                                eth_dst=dst)
14
15        if msg.buffer_id != ofproto.OFP_NO_BUFFER:
16            self.add_flow(datapath, 10, match, actions, msg.
17                           buffer_id)
18            return
19        else:
20            self.add_flow(datapath, 10, match, actions)

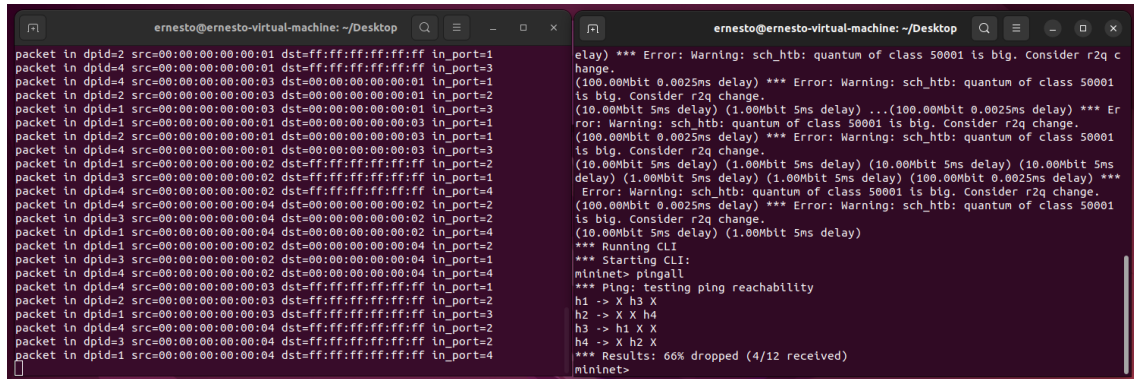
```

¹comando *OFPP_FLOOD*

3.4 Testing

3.4.1 Comando Pingall

Il risultato che ci si aspetta è un fallimento di determinate comunicazioni con conseguente drop dei pacchetti. Solo le comunicazioni tra H1H3 e H2H4 devono essere possibili

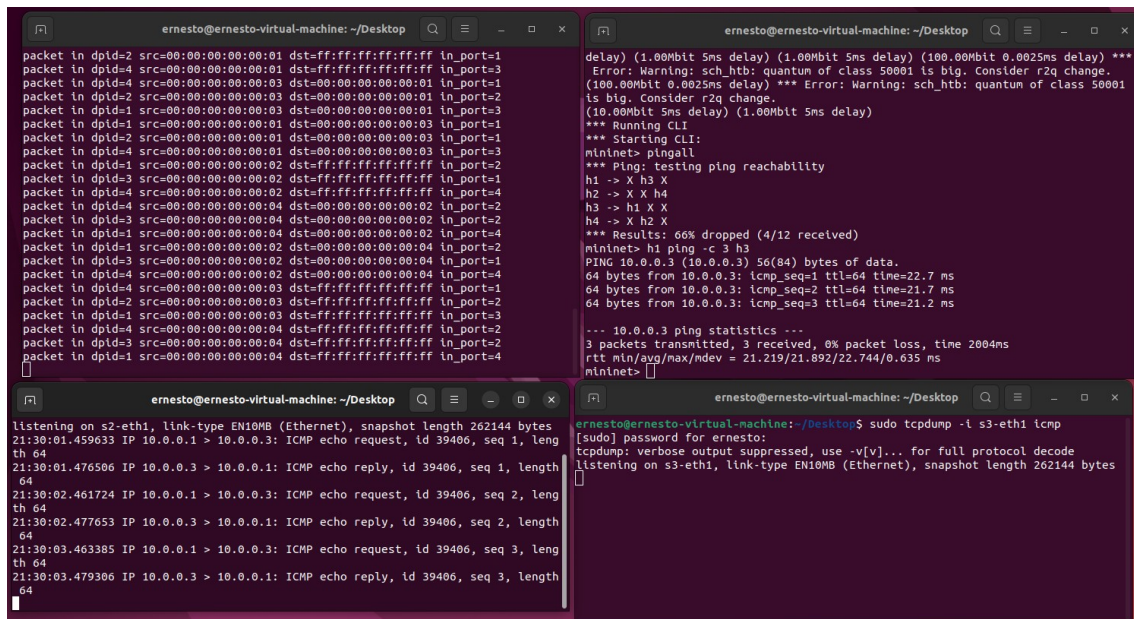


```
ernesto@ernesto-virtual-machine: ~/Desktop
packet in dpid=2 src=00:00:00:00:00:01 dst=ff:ff:ff:ff:ff:ff in_port=1
packet in dpid=4 src=00:00:00:00:00:03 dst=ff:ff:ff:ff:ff:ff in_port=3
packet in dpid=4 src=00:00:00:00:00:03 dst=00:00:00:00:00:01 in_port=1
packet in dpid=1 src=00:00:00:00:00:03 dst=00:00:00:00:00:01 in_port=3
packet in dpid=1 src=00:00:00:00:00:01 dst=00:00:00:00:00:03 in_port=1
packet in dpid=2 src=00:00:00:00:00:01 dst=00:00:00:00:00:03 in_port=1
packet in dpid=4 src=00:00:00:00:00:01 dst=00:00:00:00:00:03 in_port=3
packet in dpid=1 src=00:00:00:00:00:02 dst=ff:ff:ff:ff:ff:ff in_port=3
packet in dpid=3 src=00:00:00:00:00:02 dst=ff:ff:ff:ff:ff:ff in_port=1
packet in dpid=4 src=00:00:00:00:00:02 dst=00:00:00:00:00:02 in_port=2
packet in dpid=3 src=00:00:00:00:00:04 dst=00:00:00:00:00:02 in_port=2
packet in dpid=1 src=00:00:00:00:00:04 dst=00:00:00:00:00:02 in_port=4
packet in dpid=1 src=00:00:00:00:00:02 dst=00:00:00:00:00:04 in_port=2
packet in dpid=4 src=00:00:00:00:00:02 dst=00:00:00:00:00:04 in_port=4
packet in dpid=2 src=00:00:00:00:00:03 dst=ff:ff:ff:ff:ff:ff in_port=2
packet in dpid=1 src=00:00:00:00:00:03 dst=ff:ff:ff:ff:ff:ff in_port=3
packet in dpid=4 src=00:00:00:00:00:04 dst=ff:ff:ff:ff:ff:ff in_port=2
packet in dpid=3 src=00:00:00:00:00:04 dst=ff:ff:ff:ff:ff:ff in_port=2
packet in dpid=1 src=00:00:00:00:00:04 dst=ff:ff:ff:ff:ff:ff in_port=4

ernesto@ernesto-virtual-machine: ~/Desktop
... (100.00Mbit 0.0025ms delay) *** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
... (100.00Mbit 0.0025ms delay) *** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
... (10.00Mbit 5ms delay) (1.00Mbit 5ms delay) ... (100.00Mbit 0.0025ms delay) *** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
... (100.00Mbit 0.0025ms delay) *** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
... (10.00Mbit 5ms delay) (1.00Mbit 5ms delay) (10.00Mbit 5ms delay) (10.00Mbit 5ms delay) (1.00Mbit 5ms delay) (1.00Mbit 5ms delay) (100.00Mbit 0.0025ms delay) *** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
... (100.00Mbit 0.0025ms delay) *** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
... (10.00Mbit 5ms delay) (1.00Mbit 5ms delay)
*** Running CLI
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> X h3 X
h2 -> X X h4
h3 -> h1 X X
h4 -> X h2 X
*** Results: 66% dropped (4/12 received)
mininet>
```

Figura 2:

3.4.2 Percorso dei pacchetti nelle slice corrette



```
ernesto@ernesto-virtual-machine: ~/Desktop
packet in dpid=2 src=00:00:00:00:00:01 dst=ff:ff:ff:ff:ff:ff in_port=1
packet in dpid=4 src=00:00:00:00:00:03 dst=ff:ff:ff:ff:ff:ff in_port=3
packet in dpid=4 src=00:00:00:00:00:03 dst=00:00:00:00:00:01 in_port=1
packet in dpid=1 src=00:00:00:00:00:03 dst=00:00:00:00:00:01 in_port=3
packet in dpid=1 src=00:00:00:00:00:01 dst=00:00:00:00:00:03 in_port=1
packet in dpid=2 src=00:00:00:00:00:01 dst=00:00:00:00:00:03 in_port=1
packet in dpid=4 src=00:00:00:00:00:01 dst=00:00:00:00:00:03 in_port=3
packet in dpid=1 src=00:00:00:00:00:02 dst=ff:ff:ff:ff:ff:ff in_port=2
packet in dpid=3 src=00:00:00:00:00:02 dst=ff:ff:ff:ff:ff:ff in_port=1
packet in dpid=4 src=00:00:00:00:00:02 dst=ff:ff:ff:ff:ff:ff in_port=4
packet in dpid=1 src=00:00:00:00:00:04 dst=00:00:00:00:00:02 in_port=2
packet in dpid=3 src=00:00:00:00:00:04 dst=00:00:00:00:00:02 in_port=2
packet in dpid=1 src=00:00:00:00:00:02 dst=00:00:00:00:00:04 in_port=2
packet in dpid=4 src=00:00:00:00:00:02 dst=00:00:00:00:00:04 in_port=4
packet in dpid=4 src=00:00:00:00:00:03 dst=ff:ff:ff:ff:ff:ff in_port=1
packet in dpid=2 src=00:00:00:00:00:03 dst=ff:ff:ff:ff:ff:ff in_port=2
packet in dpid=1 src=00:00:00:00:00:03 dst=ff:ff:ff:ff:ff:ff in_port=3
packet in dpid=4 src=00:00:00:00:00:04 dst=ff:ff:ff:ff:ff:ff in_port=2
packet in dpid=3 src=00:00:00:00:00:04 dst=ff:ff:ff:ff:ff:ff in_port=2
packet in dpid=1 src=00:00:00:00:00:04 dst=ff:ff:ff:ff:ff:ff in_port=4

ernesto@ernesto-virtual-machine: ~/Desktop
listening on s2-eth1, link-type EN10MB (Ethernet), snapshot length 262144 bytes
21:30:01.459633 IP 10.0.0.1 > 10.0.0.3: ICMP echo request, id 39406, seq 1, length 64
21:30:01.476506 IP 10.0.0.3 > 10.0.0.1: ICMP echo reply, id 39406, seq 1, length 64
21:30:02.461724 IP 10.0.0.1 > 10.0.0.3: ICMP echo request, id 39406, seq 2, length 64
21:30:02.477653 IP 10.0.0.3 > 10.0.0.1: ICMP echo reply, id 39406, seq 2, length 64
21:30:03.463385 IP 10.0.0.1 > 10.0.0.3: ICMP echo request, id 39406, seq 3, length 64
21:30:03.479306 IP 10.0.0.3 > 10.0.0.1: ICMP echo reply, id 39406, seq 3, length 64

ernesto@ernesto-virtual-machine: ~/Desktop
... (10.00Mbit 5ms delay) (1.00Mbit 5ms delay) (100.00Mbit 0.0025ms delay) *** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
... (100.00Mbit 0.0025ms delay) *** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
... (10.00Mbit 5ms delay) (1.00Mbit 5ms delay)
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> X h3 X
h2 -> X X h4
h3 -> h1 X X
h4 -> X h2 X
*** Results: 66% dropped (4/12 received)
mininet> h1 ping -c 3 h3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=22.7 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=21.7 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=21.2 ms

--- 10.0.0.3 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 21.219/21.892/22.744/0.635 ms
mininet>
```

Figura 3: Ping tra H1 e H3

La corretta impostazione delle regole di slicing ha evitato la tempesta broadcast inoltrando il pacchetto solo lungo la slice corretta.

4 Service Slicing

Anche in questo caso al fine di realizzare questa tecnica di slicing è stato modificato il file originale del Controller Ryu nei seguenti punti:

```

ernesto@ernesto-virtual-machine: ~/Desktop
packet in dpid=2 src=00:00:00:00:00:01 dst=ff:ff:ff:ff:ff:ff in_port=1
packet in dpid=4 src=00:00:00:00:00:01 dst=ff:ff:ff:ff:ff:ff in_port=3
packet in dpid=4 src=00:00:00:00:00:03 dst=00:00:00:00:00:01 in_port=1
packet in dpid=2 src=00:00:00:00:00:03 dst=00:00:00:00:00:01 in_port=2
packet in dpid=1 src=00:00:00:00:00:03 dst=00:00:00:00:00:01 in_port=3
packet in dpid=2 src=00:00:00:00:00:01 dst=00:00:00:00:00:03 in_port=1
packet in dpid=4 src=00:00:00:00:00:01 dst=00:00:00:00:00:03 in_port=3
packet in dpid=2 src=00:00:00:00:00:02 dst=ff:ff:ff:ff:ff:ff in_port=1
packet in dpid=3 src=00:00:00:00:00:02 dst=ff:ff:ff:ff:ff:ff in_port=1
packet in dpid=4 src=00:00:00:00:00:02 dst=ff:ff:ff:ff:ff:ff in_port=4
packet in dpid=4 src=00:00:00:00:00:04 dst=00:00:00:00:00:02 in_port=2
packet in dpid=3 src=00:00:00:00:00:04 dst=00:00:00:00:00:02 in_port=2
packet in dpid=1 src=00:00:00:00:00:04 dst=00:00:00:00:00:02 in_port=2
packet in dpid=3 src=00:00:00:00:00:02 dst=00:00:00:00:00:04 in_port=1
packet in dpid=4 src=00:00:00:00:00:02 dst=00:00:00:00:00:04 in_port=4
packet in dpid=4 src=00:00:00:00:00:03 dst=ff:ff:ff:ff:ff:ff in_port=1
packet in dpid=2 src=00:00:00:00:00:03 dst=ff:ff:ff:ff:ff:ff in_port=2
packet in dpid=1 src=00:00:00:00:00:03 dst=ff:ff:ff:ff:ff:ff in_port=3
packet in dpid=4 src=00:00:00:00:00:04 dst=ff:ff:ff:ff:ff:ff in_port=2
packet in dpid=3 src=00:00:00:00:00:04 dst=ff:ff:ff:ff:ff:ff in_port=2
packet in dpid=1 src=00:00:00:00:00:04 dst=ff:ff:ff:ff:ff:ff in_port=4

h1 -> X h3 X
h2 -> X X h4
h3 -> h1 X X
h4 -> X h2 X
*** Results: 66% dropped (4/12 received)
mininet> h1 ping -c 3 h3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data:
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=22.7 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=21.7 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=21.2 ms

--- 10.0.0.3 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 21.219/21.892/22.744/0.635 ms
mininet> h2 ping -c 3 h4
PING 10.0.0.4 (10.0.0.4) 56(84) bytes of data:
64 bytes from 10.0.0.4: icmp_seq=1 ttl=64 time=22.9 ms
64 bytes from 10.0.0.4: icmp_seq=2 ttl=64 time=22.1 ms
64 bytes from 10.0.0.4: icmp_seq=3 ttl=64 time=21.1 ms

--- 10.0.0.4 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2001ms
rtt min/avg/max/mdev = 21.128/22.019/22.857/0.706 ms
mininet>

listening on s2-eth1, link-type EN10MB (Ethernet), snapshot length 262144 bytes
21:30:01.459633 IP 10.0.0.1 > 10.0.0.3: ICMP echo request, id 39406, seq 1, length 64
21:30:01.476506 IP 10.0.0.3 > 10.0.0.1: ICMP echo reply, id 39406, seq 1, length 64
21:30:02.461724 IP 10.0.0.1 > 10.0.0.3: ICMP echo request, id 39406, seq 2, length 64
21:30:02.477653 IP 10.0.0.3 > 10.0.0.1: ICMP echo reply, id 39406, seq 2, length 64
21:30:03.463385 IP 10.0.0.1 > 10.0.0.3: ICMP echo request, id 39406, seq 3, length 64
21:30:03.479386 IP 10.0.0.3 > 10.0.0.1: ICMP echo reply, id 39406, seq 3, length 64

tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on s3-eth1, link-type EN10MB (Ethernet), snapshot length 262144 bytes
21:31:48.006737 IP 10.0.0.2 > 10.0.0.4: ICMP echo request, id 28102, seq 1, length 64
21:31:48.023270 IP 10.0.0.4 > 10.0.0.2: ICMP echo reply, id 28102, seq 1, length 64
21:31:49.006787 IP 10.0.0.2 > 10.0.0.4: ICMP echo request, id 28102, seq 2, length 64
21:31:49.023115 IP 10.0.0.4 > 10.0.0.2: ICMP echo reply, id 28102, seq 2, length 64
21:31:50.007108 IP 10.0.0.2 > 10.0.0.4: ICMP echo request, id 28102, seq 3, length 64
21:31:50.022849 IP 10.0.0.4 > 10.0.0.2: ICMP echo reply, id 28102, seq 3, length 64

```

Figura 4: Ping tra H2 e H4

4.1 Strutture dato per gli switch

Similmente all'approccio topologico, ho definito 2 dizionari per i 4 switch. I dizionari per gli switch di transito (s2 e s3) contengono delle regole di bridging statiche, mentre per quelli agli edge della topologia contengono semplicemente informazioni circa la loro composizione, utili per definire successivamente le regole di instradamento.

```

1  EDGE_SWITCHES = {
2      1: {'local_ports': (1, 2), 'upper_port': 3, '
        lower_port': 4},
3      4: {'local_ports': (1, 2), 'upper_port': 3, '
        lower_port': 4},
4  }
5
6  RELAY_SWITCHES = {
7      2: {1: 2, 2: 1},
8      3: {1: 2, 2: 1},
9  }

```

4.2 Funzion Helper

Anche in questo caso ho definito 2 funzioni di utilità:

```

1  @staticmethod
2  def _is_video(pkt):
3      ip_pkt = pkt.get_protocol(ipv4.ipv4)
4      if ip_pkt is None:
5          return False
6      udp_pkt = pkt.get_protocol(udp.udp)
7      if udp_pkt is None:
8          return False

```

```

9         return udp_pkt.dst_port == ServiceSlicingSwitch13.
           VIDEO_UDP_PORT

1
    def _install_and_send(self, datapath, msg, priority,
2        match, actions, in_port):
3        ofproto = datapath.ofproto
4        parser = datapath.ofproto_parser
5
6        if msg.buffer_id != ofproto.OFP_NO_BUFFER:
7            self.add_flow(datapath, priority, match, actions,
8                           msg.buffer_id)
9            return
10        else:
11            self.add_flow(datapath, priority, match, actions)
12
13        data = None
14        if msg.buffer_id == ofproto.OFP_NO_BUFFER:
15            data = msg.data
16        out = parser.OFPPacketOut(datapath=datapath,
17                                   buffer_id=msg.buffer_id,
18                                   in_port=in_port, actions=
19                                   actions, data=data)
20        datapath.send_msg(out)

```

La prima funzione servirà per la classificazione del traffico (video o normale), mentre la seconda riprende semplicemente la logica di salvataggio della regola appresa e instradamento del pacchetto del codice originale. La scelta di inglobare questa logica in un metodo a parte è legata al fatto che, per realizzare lo slicing sul servizio, servirà in diversi punti del codice.

4.3 Logica di Slicing

Il codice modificato tiene conto di diverse casistiche:

Caso 1: Se il pacchetto attraversa uno switch di transito, si usa per questo una logica statica, similmente al topology slicing visto in precedenza.

```

1        if dpid in self.RELAY_SWITCHES:
2            out_port = self.RELAY_SWITCHES[dpid].get(in_port)
3            if out_port is None:
4                return
5            actions = [parser.OFPActionOutput(out_port)]
6            match = parser.OFPMatch(in_port=in_port, eth_src=
7                                   src, eth_dst=dst)
8            self._install_and_send(datapath, msg, 10, match,
9                                   actions, in_port)
10           return

```

Caso 2: Se il codice si trova in questa sezione, vuol dire che lo switch che ha attivato l'evento di ricezione è uno di edge. Il codice allora ne preleva la configurazione

e se la porta di uscita è nota e risulta essere di una dei due uplink, la logica classifica il pacchetto secondo la traccia fornita. Se invece la destinazione risulta essere l'altro host a cui lo switch è connesso, si instrada semplicemente il pacchetto. Si apprezzi inoltre l'uso di 2 priorità generiche, di modo che la regola del traffico video prevalga su quella standard.

```

1         if dst in self.mac_to_port[dpid]:
2             out_port = self.mac_to_port[dpid][dst]
3
4             if out_port in (upper_port, lower_port):
5                 out_port = upper_port if is_video else
                      lower_port
6
7             if is_video:
8                 match = parser.OFPMatch(
9                     in_port=in_port, eth_type=ether_types
10                      .ETH_TYPE_IP,
11                      ip_proto=17, eth_src=src, eth_dst=dst
12                      ,
13                      udp_dst=self.VIDEO_UDP_PORT)
14                 priority = 20 # piu' specifica e
                      prioritaria
15             else:
16                 match = parser.OFPMatch(in_port=in_port,
17                     eth_src=src, eth_dst=dst)
18                 priority = 10
19             else:
20                 # destinazione locale allo stesso switch:
21                 # consegna diretta
22                 match = parser.OFPMatch(in_port=in_port,
23                     eth_src=src, eth_dst=dst)
24                 priority = 10
25
26             actions = [parser.OFPACTIONOutput(out_port)]
27             self._install_and_send(datapath, msg, priority,
28                 match, actions, in_port)
29             return

```

Caso 3: L'ultima casistica riguarda la casistica in cui la destinazione è sconosciuta, ossia si riprende il classico MAC learning di uno switch. Al fine di impedire però il problema del flooding, è stata realizzata una logica alternativa dove il broadcast non viene totalmente realizzato per impedire le tempeste di pacchetti.

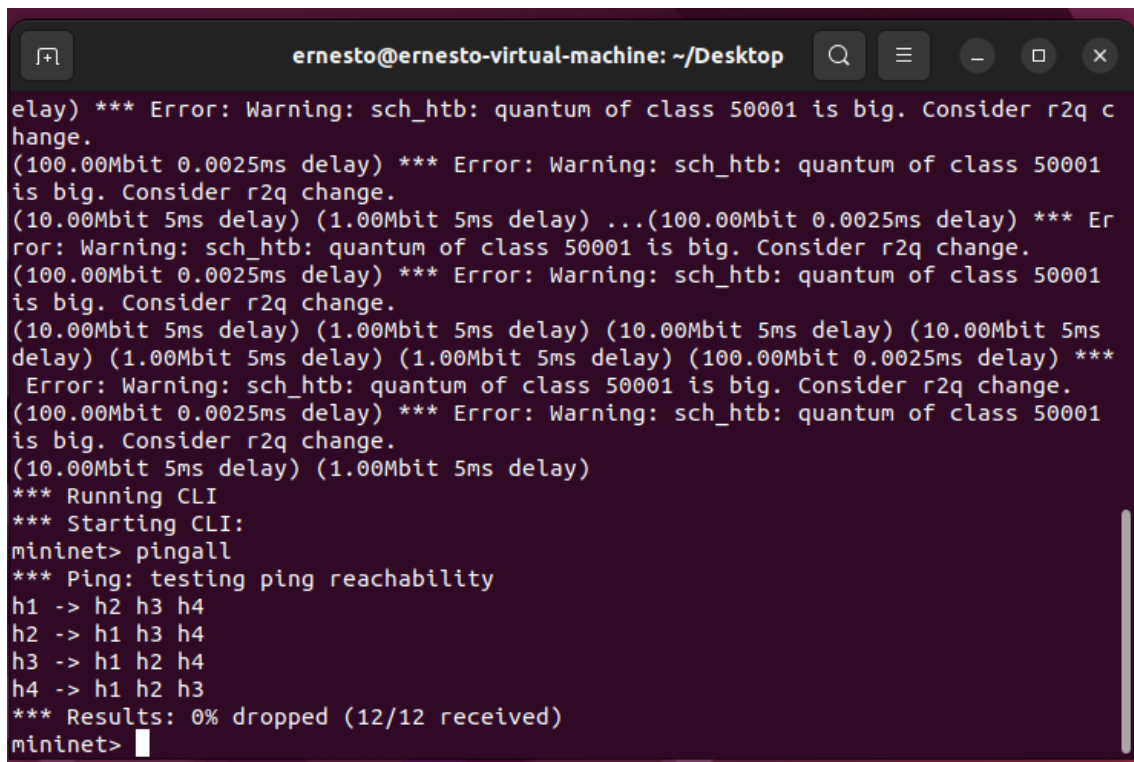
```

1         if in_port in local_ports:
2             flood_ports = [p for p in local_ports if p !=
3                 in_port] + [lower_port]
4         else:
5             flood_ports = list(local_ports)

```


4.4 Testing

4.4.1 Connettività di base

A screenshot of a terminal window titled 'ernesto@ernesto-virtual-machine: ~/Desktop'. The terminal displays a series of network simulation messages. It starts with several lines of warnings and errors from 'sch_htb' regarding the quantum of class 50001, suggesting a change to 'r2q'. These messages are interspersed with traffic delays, such as '(100.00Mbit 0.0025ms delay)' and '(10.00Mbit 5ms delay)'. After a series of these messages, the terminal shows '*** Running CLI' and '*** Starting CLI:'. The user then enters 'mininet> pingall', which triggers a 'Ping: testing ping reachability' message. This is followed by four lines showing connectivity between hosts: 'h1 -> h2 h3 h4', 'h2 -> h1 h3 h4', 'h3 -> h1 h2 h4', and 'h4 -> h1 h2 h3'. The final message is '*** Results: 0% dropped (12/12 received)', indicating successful connectivity. The prompt 'mininet>' is visible at the bottom of the terminal.

```
elay) *** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q c
hange.
(100.00Mbit 0.0025ms delay) *** Error: Warning: sch_htb: quantum of class 50001
is big. Consider r2q change.
(10.00Mbit 5ms delay) (1.00Mbit 5ms delay) ...(100.00Mbit 0.0025ms delay) *** Er
ror: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
(100.00Mbit 0.0025ms delay) *** Error: Warning: sch_htb: quantum of class 50001
is big. Consider r2q change.
(10.00Mbit 5ms delay) (1.00Mbit 5ms delay) (10.00Mbit 5ms delay) (10.00Mbit 5ms
delay) (1.00Mbit 5ms delay) (1.00Mbit 5ms delay) (100.00Mbit 0.0025ms delay) ***
Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
(100.00Mbit 0.0025ms delay) *** Error: Warning: sch_htb: quantum of class 50001
is big. Consider r2q change.
(10.00Mbit 5ms delay) (1.00Mbit 5ms delay)
*** Running CLI
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)
mininet> █
```

4.4.2 Instradamento traffico video

Come esempio si userà una comunicazione tra h1 e h3. Un analogo risultato vale per il traffico normale (simulato con traffico tcp).

```

ernesto@ernesto-virtual-machine: ~/Desktop
loading app service_slicing_controller.py
loading app ryu.controller.ofp_handler
instantiating app service_slicing_controller.py of ServiceSlicingSwitch13
instantiating app ryu.controller.ofp_handler of OFPHandler
Terminated
ernesto@ernesto-virtual-machine: ~/Desktop$ ryu-manager service_slicing_controller.py
/usr/local/lib/python3.10/dist-packages/ryu/lib/hub.py:30: EventletDeprecationWarning:
Eventlet is deprecated. It is currently being maintained in bugfix mode, and
we strongly recommend against using it for new projects.

If you are already using Eventlet, we recommend migrating to a different
framework. For more detail see
https://eventlet.readthedocs.io/en/latest/asyncio/migration.html

import eventlet
if __name__ == '__main__':
    # Block(s) were not greened, to fix this error make sure you run eventlet.monkey
    # patch() before importing any other modules.
    loading app service_slicing_controller.py
    loading app ryu.controller.ofp_handler
    instantiating app service_slicing_controller.py of ServiceSlicingSwitch13
    instantiating app ryu.controller.ofp_handler of OFPHandler

ernesto@ernesto-virtual-machine: ~/Desktop$ ryu-manager service_slicing_controller.py
/usr/local/lib/python3.10/dist-packages/ryu/lib/hub.py:30: EventletDeprecationWarning:
Eventlet is deprecated. It is currently being maintained in bugfix mode, and
we strongly recommend against using it for new projects.

If you are already using Eventlet, we recommend migrating to a different
framework. For more detail see
https://eventlet.readthedocs.io/en/latest/asyncio/migration.html

import eventlet
if __name__ == '__main__':
    # Block(s) were not greened, to fix this error make sure you run eventlet.monkey
    # patch() before importing any other modules.
    loading app service_slicing_controller.py
    loading app ryu.controller.ofp_handler
    instantiating app service_slicing_controller.py of ServiceSlicingSwitch13
    instantiating app ryu.controller.ofp_handler of OFPHandler

ernesto@ernesto-virtual-machine: ~/Desktop$ sudo tcpdump -i s3-eth1 udp port 9999
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on s3-eth1, link-type EN10MB (Ethernet), snapshot length 262144 bytes
21:57:37.842400 IP 10.0.0.1.58530 > 10.0.0.3.9999: UDP, length 1470
21:57:37.848042 IP 10.0.0.1.58530 > 10.0.0.3.9999: UDP, length 1470
21:57:37.853980 IP 10.0.0.1.58530 > 10.0.0.3.9999: UDP, length 1470
21:57:37.859550 IP 10.0.0.1.58530 > 10.0.0.3.9999: UDP, length 1470
21:57:37.864047 IP 10.0.0.1.58530 > 10.0.0.3.9999: UDP, length 1470
21:57:37.870475 IP 10.0.0.1.58530 > 10.0.0.3.9999: UDP, length 1470
21:57:37.876061 IP 10.0.0.1.58530 > 10.0.0.3.9999: UDP, length 1470
21:57:37.881610 IP 10.0.0.1.58530 > 10.0.0.3.9999: UDP, length 1470
21:57:37.887257 IP 10.0.0.1.58530 > 10.0.0.3.9999: UDP, length 1470
21:57:37.893286 IP 10.0.0.1.58530 > 10.0.0.3.9999: UDP, length 1470
21:57:37.899210 IP 10.0.0.1.58530 > 10.0.0.3.9999: UDP, length 1470
21:57:37.904150 IP 10.0.0.1.58530 > 10.0.0.3.9999: UDP, length 1470
21:57:37.909934 IP 10.0.0.1.58530 > 10.0.0.3.9999: UDP, length 1470

```

Figura 5: Il comando tcpdump mostra che il traffico sta oltrepassando solo lo switch s2, confermando l'uso della upper slice

```

ernesto@ernesto-virtual-machine: ~/Desktop
loading app service_slicing_controller.py
loading app ryu.controller.ofp_handler
instantiating app service_slicing_controller.py of ServiceSlicingSwitch13
instantiating app ryu.controller.ofp_handler of OFPHandler
Terminated
ernesto@ernesto-virtual-machine: ~/Desktop$ ryu-manager service_slicing_controller.py
/usr/local/lib/python3.10/dist-packages/ryu/lib/hub.py:30: EventletDeprecationWarning:
Eventlet is deprecated. It is currently being maintained in bugfix mode, and
we strongly recommend against using it for new projects.

If you are already using Eventlet, we recommend migrating to a different
framework. For more detail see
https://eventlet.readthedocs.io/en/latest/asyncio/migration.html

import eventlet
if __name__ == '__main__':
    # Block(s) were not greened, to fix this error make sure you run eventlet.monkey
    # patch() before importing any other modules.
    loading app service_slicing_controller.py
    loading app ryu.controller.ofp_handler
    instantiating app service_slicing_controller.py of ServiceSlicingSwitch13
    instantiating app ryu.controller.ofp_handler of OFPHandler

ernesto@ernesto-virtual-machine: ~/Desktop$ ryu-manager service_slicing_controller.py
/usr/local/lib/python3.10/dist-packages/ryu/lib/hub.py:30: EventletDeprecationWarning:
Eventlet is deprecated. It is currently being maintained in bugfix mode, and
we strongly recommend against using it for new projects.

If you are already using Eventlet, we recommend migrating to a different
framework. For more detail see
https://eventlet.readthedocs.io/en/latest/asyncio/migration.html

import eventlet
if __name__ == '__main__':
    # Block(s) were not greened, to fix this error make sure you run eventlet.monkey
    # patch() before importing any other modules.
    loading app service_slicing_controller.py
    loading app ryu.controller.ofp_handler
    instantiating app service_slicing_controller.py of ServiceSlicingSwitch13
    instantiating app ryu.controller.ofp_handler of OFPHandler

ernesto@ernesto-virtual-machine: ~/Desktop$ sudo tcpdump -i s2-eth1 tcp
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on s2-eth1, link-type EN10MB (Ethernet), snapshot length 262144 bytes
22:00:25.978434 IP 10.0.0.3.5001 > 10.0.0.1.49926: Flags [.], ack 360613, win 72
5, options [nop,nop,TS val 271232903 ecr 1605859413], length 0
22:00:25.987042 IP 10.0.0.1.49926 > 10.0.0.3.5001: Flags [.], seq 360613:362061,
ack 29, win 83, options [nop,nop,TS val 1605859437 ecr 271232188], length 1448
22:00:25.999154 IP 10.0.0.1.49926 > 10.0.0.3.5001: Flags [.], seq 362061:363509,
ack 29, win 83, options [nop,nop,TS val 1605859437 ecr 271232188], length 1448
22:00:26.011276 IP 10.0.0.1.49926 > 10.0.0.3.5001: Flags [P.], seq 363509:366405,
ack 29, win 83, options [nop,nop,TS val 1605859461 ecr 271232212], length 2896
22:00:26.014590 IP 10.0.0.3.5001 > 10.0.0.1.49926: Flags [.], ack 363509, win 72
5, options [nop,nop,TS val 271232939 ecr 1605859437], length 0
22:00:26.026819 IP 10.0.0.3.5001 > 10.0.0.1.49926: Flags [.], ack 366405, win 72
5, options [nop,nop,TS val 271232951 ecr 1605859461], length 0
22:00:26.035508 IP 10.0.0.1.49926 > 10.0.0.3.5001: Flags [P.], seq 366405:369301,
ack 29, win 83, options [nop,nop,TS val 1605859486 ecr 271232237], length 2896

```